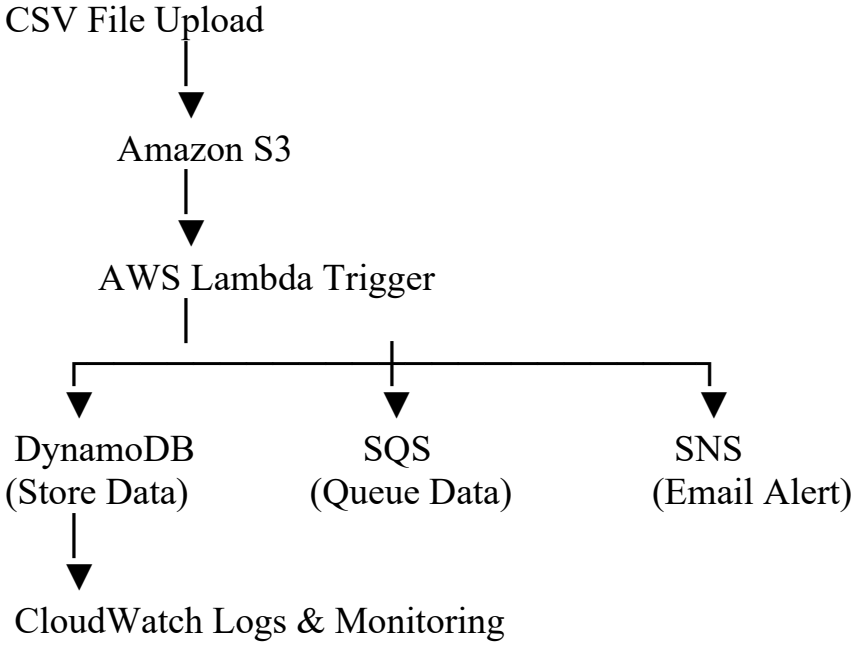
Project Title

Scalable Serverless Data Processing and Real-Time Business Analytics Platform Using AWS

AWS Services Used

- Amazon S3
- AWS Lambda
- Amazon SQS
- DynamoDB
- CloudWatch
- SNS

ARCHITECTURE FLOW



PROJECT ABSTRACT:

This project demonstrates a scalable and serverless data processing architecture using AWS services. The solution automatically processes CSV files uploaded to Amazon S3, triggers AWS Lambda for data transformation, stores processed records in DynamoDB, sends messages to Amazon SQS for queue-based processing, generates notifications using Amazon SNS, and tracks execution logs through Amazon CloudWatch.
The architecture eliminates server management, improves scalability, minimizes operational overhead, and provides real-time business analytics capabilities.

Step 1: Create S3 Bucket

Purpose
Store incoming business data files (CSV/JSON).

Procedure

1. Login to AWS Console.
2. Open **Amazon S3**.
3. Click **Create Bucket**.
4. Bucket Name: “business-data-lake”
5. Keep default settings.
6. Create bucket.

Upload Sample File



sales_data.csv

OrderID	Customer	Amount
1001	Ram	50,000
1002	Kumar	500
1003	John	1,500
1004	Tamil	2,50,000

Output

1. S3 Bucket Created Successfully
2. business-data-lake
3. | └——— sales_data.csv

Step 2: Create DynamoDB Table
Purpose: Store processed records.
Procedure:

1. Search **DynamoDB**
2. Click **Create Table**

Enter:

Field	Value
Table Name	SalesData
Partition Key	OrderID
Data Type	String

3. Keep defaults
4. Click **Create Table**

Wait until table becomes **Active**.

Step 3: Create SQS Queue
Purpose

Store processing messages.

Procedure

1. Search **SQS**
2. Create Queue

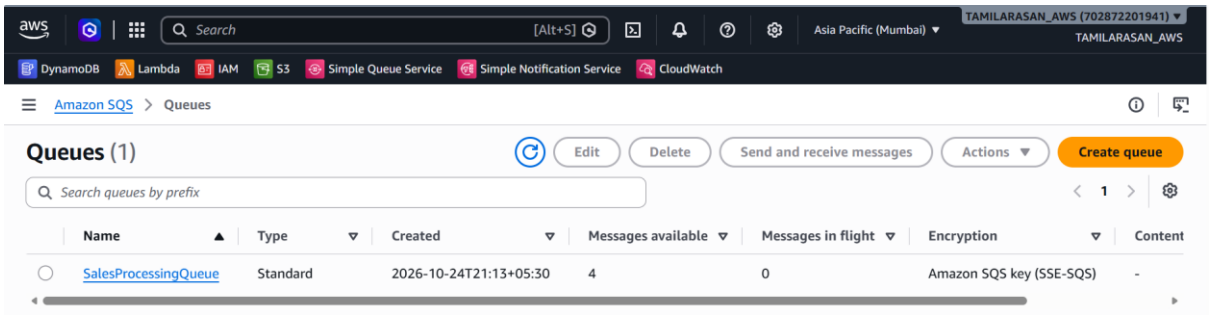
Enter:

Field	Value
Queue Type	Standard
Queue Name	SalesProcessingQueue

3. Click **Create Queue**

Copy Queue URL

Save: https://sqs-region.amazonaws.com/accountid/SalesProcessingQueue



Step 4: Create SNS Topic

Purpose

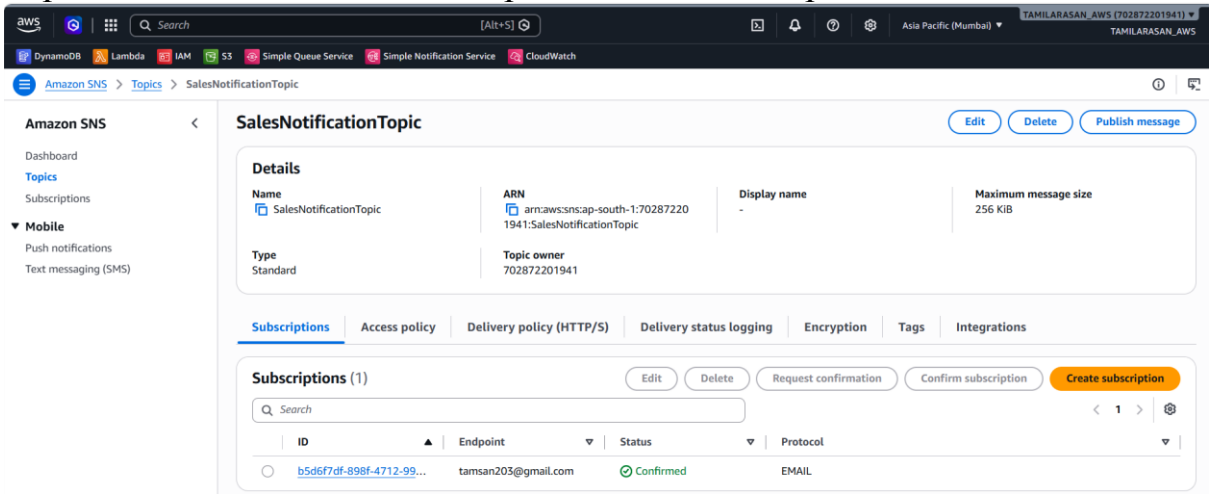
Send email notifications.

Procedure

1. Search **SNS**
2. Click **Topics**
3. Create Topic

Select: Standard Topic

Topic Name: SalesNotificationTopic Click: Create Topic



Step 5: Subscribe Email to SNS

1. Open created topic
2. Click **Create Subscription**

Enter:

Field	Value
Protocol	Email
Endpoint	tamsan203@gmail.com

3. Click **Create Subscription**
4. Check inbox
5. Click **Confirm Subscription**

Status becomes: Confirmed

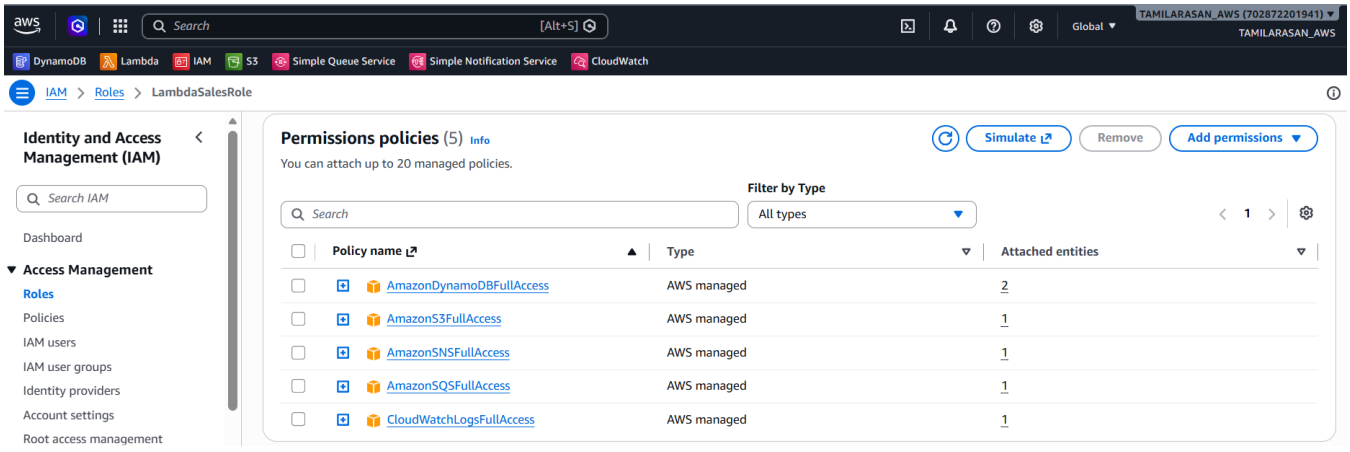
Step 6: Create IAM Role for Lambda

Purpose: Allow Lambda to access S3, DynamoDB, SQS, SNS and CloudWatch.

Procedure

1. Search IAM
2. Roles
3. Create Role

Select: AWS Service
Use Case: Lambda
Attach policies:
AmazonS3FullAccess
AmazonDynamoDBFullAccess
AmazonSQSFullAccess
AmazonSNSFullAccess
CloudWatchLogsFullAccess
Role Name: LambdaSalesRole → Create Role.



Step 7: Create Lambda Function

Procedure

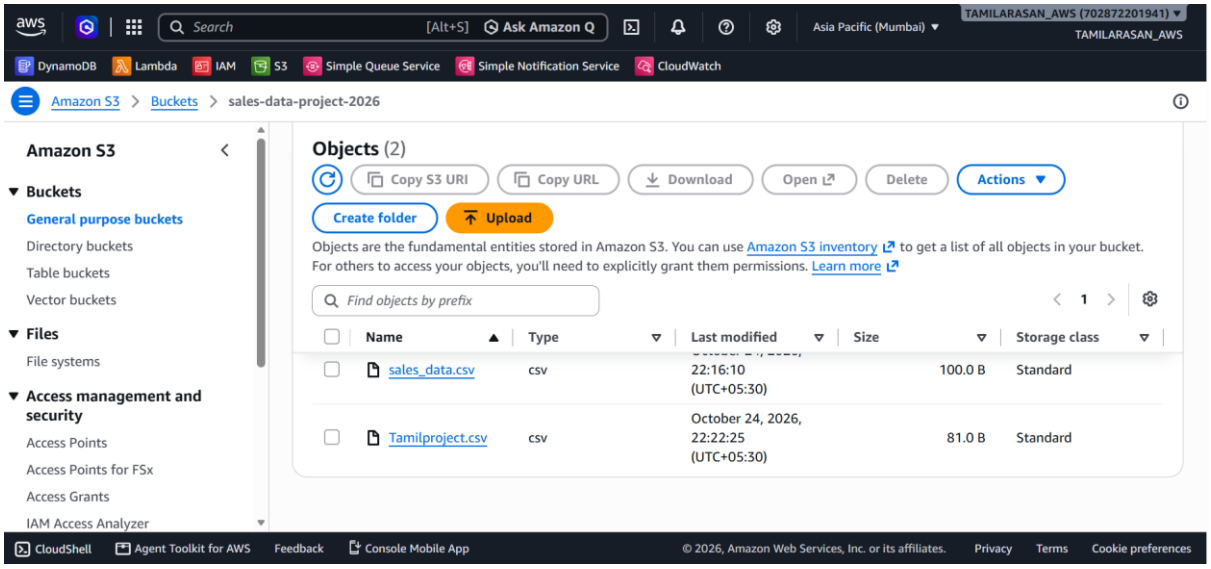
1. Search Lambda
2. Create Function

Select: Author from Scratch

Values:

Field	Value
Function Name	ProcessSalesData
Runtime	Python 3.13
Execution Role	LambdaSalesRole

Click **Create Function**



Step 8: Lambda Code

Open Lambda → Code Tab

Replace existing code:Python

```
import json
import boto3
import csv

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('SalesData')

sqs = boto3.client('sqs')
sns = boto3.client('sns')

QUEUE_URL = ' https://sqs.ap-south-1.amazonaws.com/702872201941/SalesProcessingQueue'

TOPIC_ARN = ' arn:aws:sns:ap-south-1:702872201941:SalesNotificationTopic '

def lambda_handler(event, context):

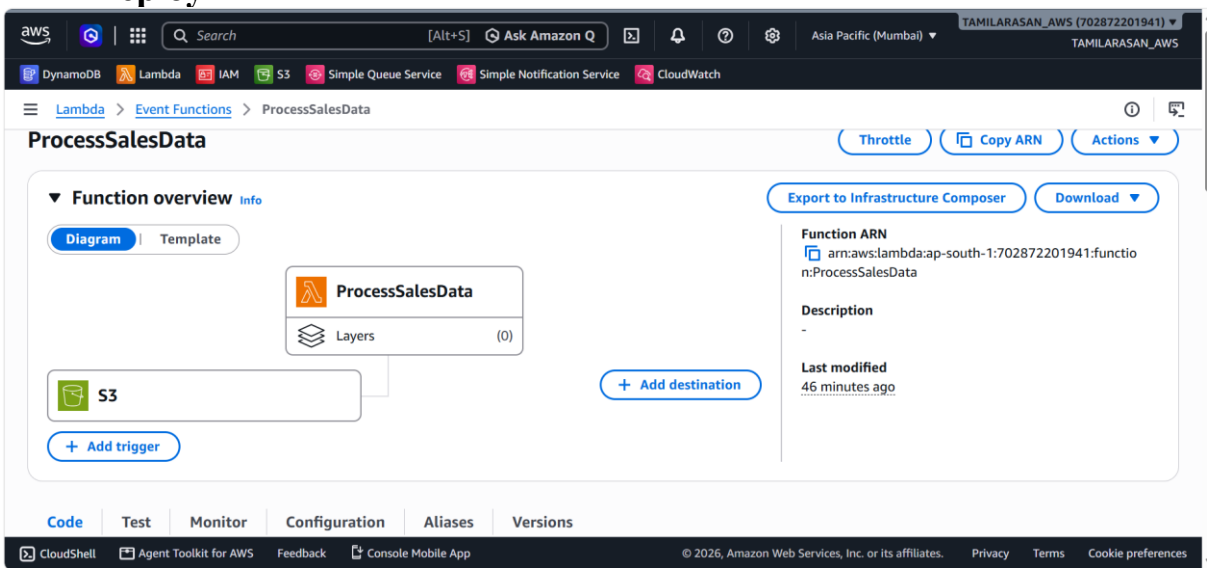
for record in event['Records']:

bucket = record['s3']['bucket']['name']
key = record['s3']['object']['key']
```

```
s3 = boto3.client('s3')

response = s3.get_object(
Bucket=bucket,
Key=key
)
content = response['Body'].read().decode('utf-8')
reader = csv.DictReader(content.splitlines())
for row in reader:
table.put_item(
Item={
'OrderID': row['OrderID'],
'Customer': row['Customer'],
'Amount': row['Amount']
}
)
sqs.send_message(
QueueUrl=QUEUE_URL,
MessageBody=json.dumps(row)
)
sns.publish(
TopicArn=TOPIC_ARN,
Subject='Sales File Processed',
Message='Sales data successfully processed'
)
return {
'statusCode': 200
}
```

Click **Deploy**.



Step 9: Configure S3 Trigger

1. Open Lambda
2. Configuration
3. Triggers
4. Add Trigger

Choose: S3
Select bucket: sales-data-project-2026
Event Type: PUT
Click: Add
Now every file upload triggers Lambda automatically.

Step 10: Configure CloudWatch Monitoring

1. Search CloudWatch
2. Logs
3. Log Groups

You will see: /aws/lambda/ProcessSalesData
Open it to monitor:

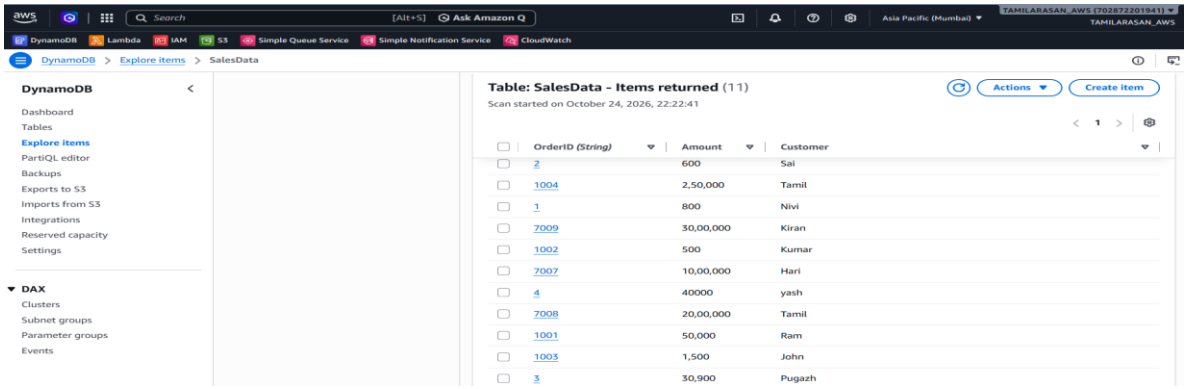
- Invocations
- Errors
- Duration
- Success Logs

Step 11: Test End-to-End
Upload CSV file to S3.
What happens?

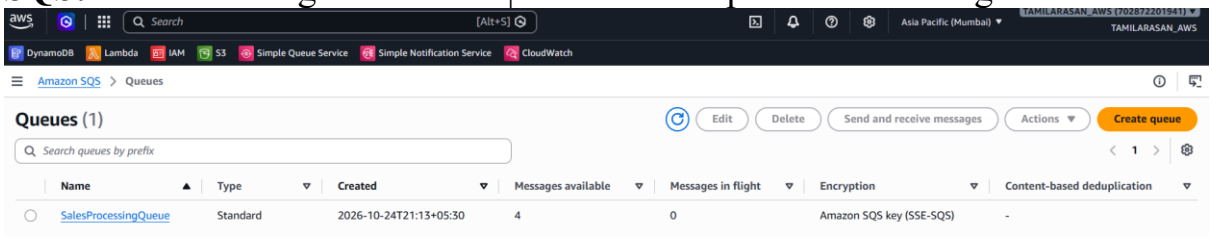
✅ S3 receives file
✅ Lambda automatically runs
✅ Data inserted into DynamoDB
✅ Message sent to SQS
✅ Email notification sent through SNS
✅ Logs generated in CloudWatch

Step 12: Verify Output
DynamoDB
SalesData table:

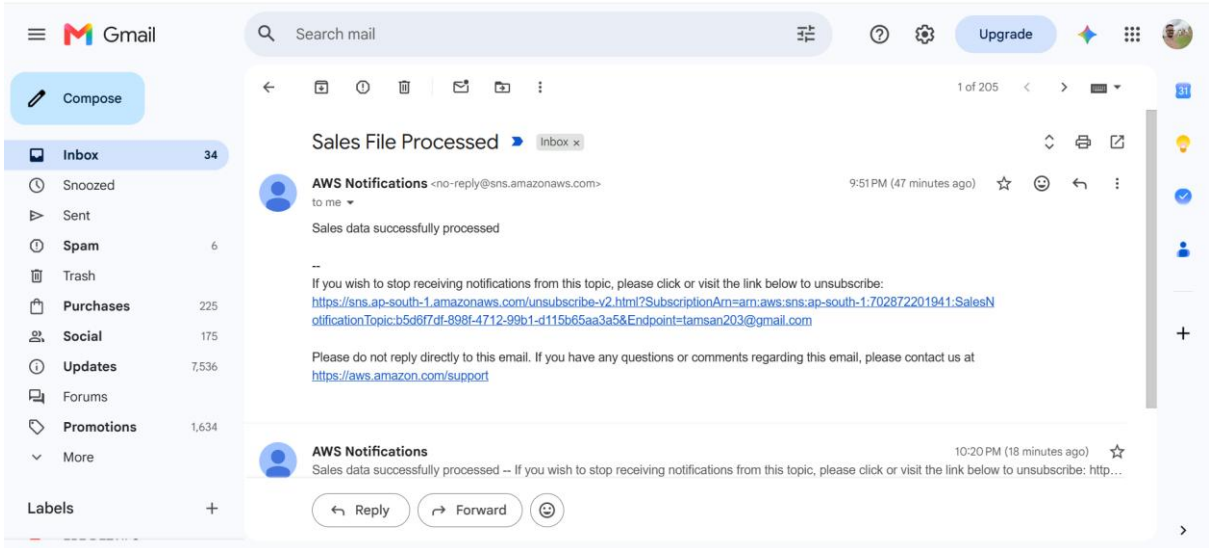
OrderID	Customer	Amount
1001	Ravi	2500
1002	Kumar	3200
1003	Raj	1800



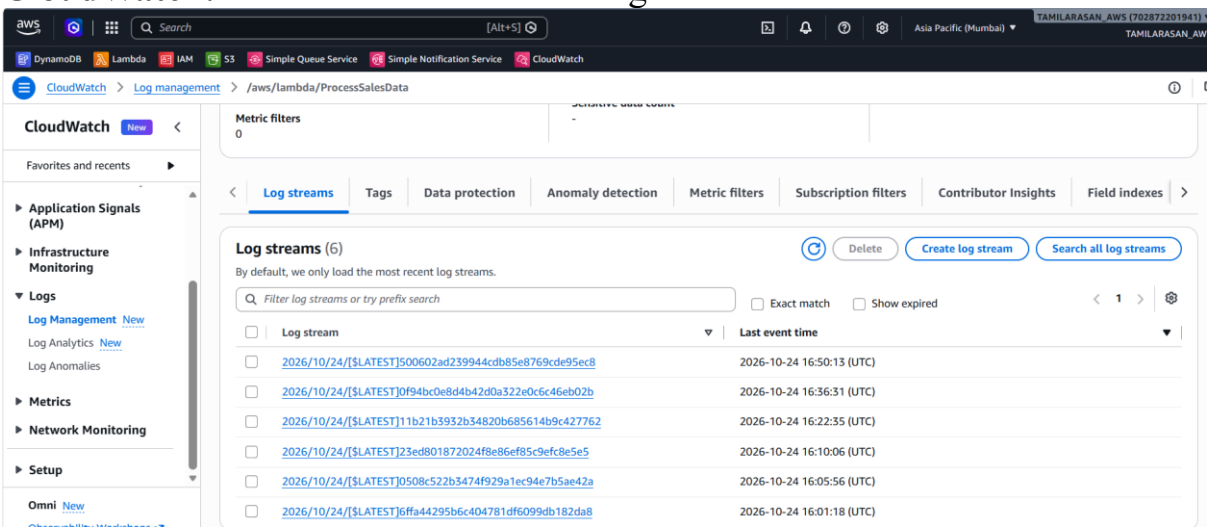
SQS: Check: Messages Available | Should show processed messages.



SNS: You receive email: Sales File Processed| Sales data successfully processed



CloudWatch: View Lambda execution logs.



Actual Result:
All services integrated successfully and processed uploaded data without manual intervention. The workflow achieved end-to-end serverless automation.

Troubleshooting Notes
DynamoDB Shows Old Records
Cause:
Deleting files from S3 does not remove records from DynamoDB.
Resolution:

- Delete records manually.
- Recreate table.
- Configure TTL if required.

Lambda Not Triggering
Verify:

- S3 trigger configuration
- IAM permissions
- CloudWatch logs

SNS Email Not Received
Verify:

- Email subscription status
- Topic ARN
- SNS permissions

DynamoDB Empty
Verify:

- Lambda execution
- CSV format
- Table name
- IAM permissions

---

10. Conclusion
This project successfully demonstrates a modern serverless architecture using AWS services. The solution automatically processes incoming business data, stores structured information, generates alerts, and provides monitoring capabilities without managing servers. By combining Amazon S3, Lambda, DynamoDB, SQS, SNS, and CloudWatch, the platform delivers a scalable, reliable, and cost-effective real-time business analytics solution suitable for enterprise data processing workloads.